Advanced LLM Chatbot

AI Systems Engineering

MUHAMMAD ABUZAR

MATRICOLA: D03000198

SUBJECT: AI SYSTEM ENGINEERING

PROF. ROBERTO PIATRANTUNIO

GITHUB LINK: https://github.com/YOUR-USERNAME/llm-chatbot-ai-systems-engineering

Single integrated LLM chatbot covering Transformer architecture, attention, SSL, fine-tuning, few-shot prompting, RAG, vector database, MCP-style tools, context engineering, agents and preference alignment.

1. Executive Summary

This project develops one integrated LLM-powered chatbot. It demonstrates the engineering path from Transformer architecture and attention through self-supervised learning, Transformer fine-tuning, few-shot prompting, RAG, vector search, MCP-style tools, context engineering, agents and preference alignment.

2. Requirements Analysis

The chatbot must answer user questions, use project knowledge when appropriate, retrieve relevant information, support tool/context access and run locally. The implementation follows the course project lifecycle: requirements analysis, design, prototype development, continuous testing and local deployment.

3. System Design

The main runtime pipeline is:

User → Context Engineering → Agent → RAG / Vector Database / MCP Tool → Few-shot Prompt → Transformer LLM → Response

The training lifecycle is:

Pre-trained Transformer → Self-Supervised Learning → Instruction Fine-Tuning → Preference Alignment → Deployment

4. Transformer and Large Language Model

The chatbot uses a Transformer-based language model. A separate transparent attention implementation demonstrates the mathematical core without replacing the chatbot itself.

5. Attention Mechanism

Scaled dot-product attention is implemented as:

Attention(Q,K,V) = softmax(QK■ / √d■)V

The code calculates attention scores, normalized attention weights and the weighted representation.

6. Self-Supervised Learning

The SSL stage uses unlabeled chatbot-domain text. The provided recipe describes masked language modeling, where masked tokens become the training signal.

7. Transformer Fine-Tuning

An instruction dataset is provided. The fine-tuning module documents a supervised instruction-tuning configuration. Actual model training should only be reported after execution.

8. Few-Shot Prompting

The context engine adds example user/assistant interactions to the prompt. These examples guide the model toward the desired response style and behavior.

9. RAG

When a question concerns the chatbot's technical knowledge, the agent can retrieve relevant documents and place them into the prompt before generation.

10. Vector Database

The prototype implements a lightweight TF-IDF vector store with cosine similarity. Its interface can later be replaced with a production vector database such as FAISS or Qdrant.

11. MCP

The project contains an educational MCP-style structured tool/context interface. The agent calls a defined knowledge tool and receives structured context and source names.

12. Context Engineering

The final context combines system instructions, few-shot examples, retrieved knowledge, tool context and the current user question.

13. Agents

The agent performs a routing decision: direct response or knowledge retrieval. It then executes the selected action before generation.

14. Reinforcement / Preference Alignment

The alignment module demonstrates a reward comparison between chosen and rejected responses. A full preference-optimization method such as DPO can be used as a training extension. No completed large-scale RL run is claimed by this repository.

15. Reasoning and Chain-of-Thought

The chatbot supports reasoning-oriented planning, such as deciding whether retrieval or a tool is needed. Private chain-of-thought is not exposed or stored. The system instead returns concise conclusions and source/tool information.

16. Prototype Development

The repository provides a CLI chatbot and FastAPI API. It uses `google/flan-t5-small` when the Transformers model weights are available locally.

17. Continuous Testing

Automated tests cover vector search, RAG, MCP-style context retrieval, prompt construction and the alignment reward function. The API also provides a health endpoint.

18. Local Deployment

A Dockerfile is provided for local container deployment. Uvicorn can run the FastAPI service directly.

19. Reproducibility

The repository contains source code, requirements, datasets, knowledge-base files, tests, README instructions and documented training stages.

20. Limitations

The vector database is a lightweight educational implementation. SSL, fine-tuning and preference-alignment stages require appropriate model weights and compute for actual training. The MCP component is explicitly educational rather than a claim of production MCP-server completeness.

21. Conclusion

The result is one coherent chatbot system covering the requested LLM engineering pipeline from Transformer and attention fundamentals to RAG, vector retrieval, MCP-style tools, context engineering, agents and preference alignment.